



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO

DIPARTIMENTO DI INFORMATICA

CORSO DI LAUREA IN INFORMATICA

TESI DI LAUREA

IN

CYBERSECURITY

Rilevamento di Fileless Malware tramite Machine Learning: un approccio di Analisi Forense

RELATORE:

Prof. Christian Catalano

LAUREANDO:

Nicolas Bellomo

ANNO ACCADEMICO 2025 - 2026

Indice

Introduzione	1
1 Stato dell'Arte e Meccanismi di Persistenza	3
1.1 Anatomia di un'Infezione Fileless	3
1.2 Abuso di Binari Legittimi (LOLBins)	4
1.3 Il Registro di Sistema e il Payload Storing	5
1.3.1 Analisi delle Chiavi di Persistenza	6
1.3.2 Persistenza tramite Windows Management Instrumentation (WMI)	7
1.4 Limiti dell'Incident Response nello Scenario Fileless	7
2 Algoritmo di rilevamento per script PowerShell malevoli	9
2.1 Contesto e Scelte Progettuali	9
2.2 Classificazione tramite Algoritmo Random Forest	10
2.2.1 Selezione e Preparazione del Dataset	10
2.2.2 Estrazione delle Caratteristiche (Feature Engineering)	11
2.2.3 Ottimizzazione del Modello e Metrica F2	15
2.2.4 Validazione e Risultati Finali	16
2.2.5 I Limiti dell'Intelligenza Artificiale e il Sistema Ibrido	17
3 Implementazione del Tool di Analisi Forense	19
3.1 Acquisizione degli Artefatti di Sistema	20
3.1.1 Cronologia dei Comandi (Console History)	20
3.1.2 Event Logging Avanzato (Event ID 4104)	21
3.1.3 L'artefatto di Persistenza: NTUSER.DAT e Live Forensics	22
3.2 Parsing e Normalizzazione dei Dati	23

3.2.1	Elaborazione dell'Event Log (EVTX)	23
3.2.2	Ottimizzazione della Leggibilità per l'Analisi Manuale	24
3.3	Design dell'Interfaccia e Sincronizzazione dei Processi	24
3.3.1	Progettazione della Dashboard Principale	26
3.4	Esempio di Risultati dell'Analisi	27
4	Conclusioni e Sviluppi Futuri	31
4.1	Conclusioni	31
4.2	Sviluppi Futuri	32
	Bibliografia	35

Introduzione

Nel panorama odierno della cybersicurezza si assiste a un cambiamento radicale nelle strategie di infezione: il progressivo abbandono dei tradizionali file eseguibili malevoli a favore di un approccio *fileless* [14]. Come suggerisce il termine, questa tipologia di minaccia non deposita file eseguibili (.exe) sospetti sul disco rigido, ma opera in modo silente sfruttando la memoria RAM e gli strumenti di amministrazione legittimi già presenti nel sistema operativo Windows (tecnica nota come "Living off the Land"). Tra questi, PowerShell si è affermato come il vettore di attacco d'elezione, grazie alla sua profonda integrazione con il sistema e alla capacità di eseguire codice complesso direttamente in memoria.

Questa evoluzione nasce dalla precisa esigenza degli attaccanti di eludere le soluzioni di sicurezza convenzionali. Gli antivirus classici, basati su firme virali (*signature-based*), focalizzano la loro attività di scansione prevalentemente sul file system. Di conseguenza, un payload malevolo iniettato tramite PowerShell, spesso pesantemente offuscato o codificato, risulta di fatto invisibile ai tradizionali perimetri di difesa [14].

Tuttavia, operare esclusivamente in RAM presenta un limite critico: la volatilità. Per sopravvivere ai riavvii della macchina, la minaccia è costretta a stabilire un meccanismo di persistenza, abusando di componenti legittimi come il Registro di Sistema (es. chiavi Run all'interno del file NTUSER.DAT) per garantire la ri-esecuzione automatica dei comandi malevoli [10].

È esattamente in questo complesso scenario che si inserisce il presente lavoro di tesi. L'obiettivo principale del progetto è la progettazione e lo sviluppo di un framework di *Digital Forensics* e *Incident Response* specificamente ideato per automatizzare l'identificazione di abusi di PowerShell.

A differenza dei sistemi di prevenzione in tempo reale, il software sviluppato

opera in una logica di indagine *post-mortem*. Il vero nucleo innovativo del progetto risiede nell'integrazione di un motore di Machine Learning, basato sull'algoritmo *Random Forest*, capace di classificare automaticamente i comandi estratti come legittimi o malevoli.

Il cuore del sistema risiede nella capacità di analizzare e classificare artefatti forensi cruciali, come le tracce della *Console History* e, in particolare, i log operativi avanzati (*Script Block Logging* - Event ID 4104), capaci di catturare il codice PowerShell nella sua forma de-offuscata.

Attraverso l'applicazione di algoritmi di Machine Learning, il sistema valuta il rischio dei payload contenuti in tali artefatti, assegnando uno score di pericolosità che guida l'investigatore nell'identificazione delle minacce concrete. Questo approccio fornisce agli analisti forensi un efficace strumento di supporto decisionale, permettendo di ottimizzare i tempi di indagine, smascherare le tecniche di elusione e ricostruire con precisione le attività malevole eseguite esclusivamente nella memoria volatile del sistema.

1. Stato dell'Arte e Meccanismi di Persistenza

1.1 Anatomia di un'Infezione Fileless

L'esecuzione di un malware *fileless* è ingegnerizzata per azzerare l'impronta sul file system tradizionale. Il ciclo di vita di queste minacce si condensa tipicamente in tre fasi operative rapide e silenziose:

1. **Compromissione Iniziale:** Il punto di ingresso è generalmente rappresentato da due vettori principali. Il primo sfrutta l'interazione dell'utente: file esca (come documenti Office) che, aggirando tramite ingegneria sociale difese preventive come la *Visualizzazione Protetta* (*Protected View*), inducono la vittima ad abilitare macro malevole. Il secondo, più silente e pericoloso, prevede lo sfruttamento diretto di vulnerabilità del sistema operativo o di servizi di rete esposti (tecniche di *exploiting*). In entrambi i casi, l'obiettivo finale non è scrivere un binario su disco, ma richiamare PowerShell in background, iniettando le prime istruzioni d'attacco direttamente nello spazio di memoria di un processo nativo.
2. **Chiamata di Rete (C2):** Sfruttando oggetti nativi di sistema (come la classe `Net.WebClient`), lo script iniziale scarica il codice malevolo di secondo stadio da un server remoto controllato dall'attaccante.
3. **Esecuzione In-Memory:** Attraverso tecniche di *Reflection*, il payload finale viene iniettato direttamente nella memoria RAM ed eseguito. Poiché il ciclo di vita del malware è interamente legato al processo ospite, l'attacco non genera alcun file eseguibile sul disco rigido.

Per contrastare questa specifica tipologia di attacchi volatili, Microsoft ha integrato nelle versioni moderne di Windows l'Antimalware Scan Interface (AMSI) [11, 7]. Si tratta di uno standard di sicurezza progettato per ispezionare gli script (come quelli PowerShell o VBScript) direttamente nella memoria RAM, un attimo prima della loro esecuzione, analizzando il codice in chiaro indipendentemente dal livello di offuscamento iniziale.

Tuttavia, l'architettura dei malware *fileless* più sofisticati prevede una contro-misura dedicata: sfruttando le medesime tecniche di *Reflection* utilizzate per l'esecuzione, il *payload* è spesso istruito per alterare le variabili interne di PowerShell e "accecare" il sistema di controllo (*AMSI Bypass*). Disabilitando questo sensore a livello di processo, il malware rende inefficaci i controlli degli antivirus tradizionali, proseguendo l'infezione in totale invisibilità.

1.2 Abuso di Binari Legittimi (LOLBins)

Per portare a termine le fasi appena descritte, il paradigma Living Off The Land (LOTL) prevede l'utilizzo esclusivo di strumenti già preinstallati nel sistema operativo [6]. Questi file eseguibili nativi, noti come LOLBins (Living Off The Land Binaries), godono della totale fiducia del sistema e aggirano agilmente i meccanismi di Whitelisting. Tra i più sfruttati dalle minacce fileless troviamo:

- ***powershell.exe***: È il framework d'elezione per il malware fileless poiché integra nativamente il framework .NET e le API di sistema. La sua potenza risiede nell'esecuzione in-memory: può scaricare, decifrare e avviare payload complessi direttamente nella RAM, senza che alcun file tocchi mai il disco, evitando così la scansione degli antivirus tradizionali [5].
- ***cmd.exe***: Viene utilizzato come interprete di comandi per avviare istruzioni concatenate tramite il parametro */c*. Questo consente di eseguire una sequenza compatta di operazioni (come l'esfiltrazione di dati e il download di payload) e di terminare immediatamente il processo, contribuendo all'occultamento dell'attività malevola [9].

- ***mshhta.exe***: Gestisce le HTML Applications (.hta). Se richiamato tramite una chiave di registro, può eseguire script JavaScript o VBScript ospitati su server esterni senza creare file locali, aggirando i controlli di sicurezza [9].
- ***regsvr32.exe***: Strumento destinato alla registrazione delle DLL, viene abusato per scaricare ed eseguire script remoti in memoria. Essendo un binario Microsoft regolarmente firmato, il suo utilizzo elude i sistemi di controllo basati su whitelist applicative [9].
- ***rundll32.exe***: Progettato per caricare funzioni contenute in librerie dinamiche, viene sfruttato dai malware per richiamare DLL di sistema e forzarle a interpretare codice malevolo mascherandolo all'interno di un processo affidabile [9].
- ***certutil.exe***: Utile per la gestione dei certificati digitali, è frequentemente utilizzato per convertire stringhe codificate in Base64 (precedentemente memorizzate nel Registro) in payload eseguibili in memoria [9].

Sebbene esistano molteplici LOLBins, l'assoluta versatilità e potenza di esecuzione di PowerShell lo hanno reso il fulcro indiscusso delle moderne catene di attacco fileless. Per questo motivo, l'estrazione, l'analisi e la classificazione automatizzata dei suoi script rappresentano il focus centrale del framework investigativo sviluppato in questo lavoro di tesi.

1.3 Il Registro di Sistema e il Payload Storing

Poiché la RAM si svuota a ogni spegnimento della macchina, per garantire la sopravvivenza al riavvio senza lasciare file sospetti sul disco, il malware fileless trasforma il Registro di Sistema di Windows da un semplice database di configurazione a un vero e proprio supporto di storage invisibile [10, 14].

Questa tecnica, nota come *registry-only persistence*, si articola solitamente in due fasi principali di *Payload Storing*:

1. **Frammentazione e Offuscamento**: Il codice malevolo (es. script PowerShell) viene codificato in formato Base64 e frammentato in più valori di registro per eludere i limiti dimensionali delle singole chiavi.

2. **Mimetismo nei Dati:** Il payload frammentato viene salvato all'interno di chiavi dai nomi apparentemente legittimi o in aree del registro poco monitorate (ad esempio sottochiavi legate alla telemetria o in `HKCU\Software\Classes`).

Al riavvio della macchina, il sistema operativo o altri processi legittimi fungono da innesco (*trigger*), prelevando queste stringhe testuali, ricomponendole e iniettandole direttamente nella memoria RAM. L'unico modo efficace per intercettare questo payload prima che la CPU lo esegua è attraverso funzionalità di logging avanzate, come lo *Script Block Logging* (Event ID 4104), capace di catturare le istruzioni de-offuscate in tempo reale.

1.3.1 Analisi delle Chiavi di Persistenza

Il malware sfrutta diverse chiavi HKEY (acronimo di *Handle to a Key*) per forzare l'esecuzione automatica del proprio codice frammentato. In particolare, gli attaccanti abusano degli *hive* HKLM (`HKEY_LOCAL_MACHINE`) per garantire la persistenza all'avvio dell'intero sistema, e HKCU (`HKEY_CURRENT_USER`) per attivare subdolamente l'infezione al logon dello specifico utente [14]. La Tabella 1.1 riassume i principali punti di abuso:

Chiave di Registro (Path Completo)	Funzione / Abuso
<code>HKCU\Software\Microsoft\Windows\CurrentVersion\Run</code>	Persistenza utente al logon [12]
<code>HKLM\Software\Microsoft\Windows\CurrentVersion\Run</code>	Persistenza globale al logon [12]
<code>HKLM\Software\Microsoft\WindowsNT\CurrentVersion\Winlogon\Userinit</code>	Esecuzione codice in fase iniziale [12]
<code>HKLM\Software\Microsoft\WindowsNT\CurrentVersion\ImageFileExecutionOptions</code>	Hijacking dell'esecuzione [12]

Tabella 1.1. Principali chiavi di registro sfruttate dai malware fileless [14, 12].

1.3.2 Persistenza tramite Windows Management Instrumentation (WMI)

Un ulteriore e sofisticato vettore di persistenza sfruttato dalle minacce *fileless* è il Windows Management Instrumentation (WMI). Il WMI è l'infrastruttura nativa e legittima di Microsoft per la gestione dei dati, delle configurazioni e delle operazioni di sistema.

Gli attaccanti abusano di questa architettura per ottenere l'esecuzione automatica del codice senza dover scrivere alcun file su disco, sfruttando il meccanismo di sottoscrizione agli eventi [13]. Tale tecnica prevede la creazione silente di tre componenti logici:

- **Event Filter:** Un filtro che si iscrive a specifici eventi di sistema (ad esempio, l'avvio della macchina o un determinato orario).
- **Event Consumer:** Il contenitore del *payload* malevolo, che in questo contesto è quasi sempre uno script PowerShell offuscato.
- **FilterToConsumerBinding:** L'oggetto logico che lega il verificarsi dell'evento (il filtro) all'esecuzione del codice (il consumer).

Poiché i *payload* WMI vengono archiviati direttamente all'interno del *repository* di sistema (tipicamente nel file `OBJECTS.DATA`) e la loro esecuzione è affidata a un processo di sistema legittimo (`WmiPrvSE.exe`), questa tecnica aggira facilmente le classiche firme degli antivirus [6, 13]. Lo *stealth* garantito dal WMI costringe l'analista forense a interrogare direttamente il *repository* con strumenti specializzati per individuare i *consumer* anomali e isolare i comandi iniettati.

1.4 Limiti dell'Incident Response nello Scenario Fileless

L'assenza di un file eseguibile altera radicalmente il paradigma forense, spostando la ricerca delle evidenze verso componenti dinamici come i log degli eventi e le chiavi di Registro [8]. Questa indagine si scontra tuttavia con severi ostacoli architetturali.

Il primo ostacolo è la **fragilità temporale del dato**. La ricostruzione dell'attacco dipende dal recupero dei comandi in chiaro (Event ID 4104). Poiché i registri eventi di Windows hanno una capacità limitata e, in molte configurazioni, sovrascrivono automaticamente gli eventi meno recenti una volta raggiunta la dimensione massima, un elevato volume di logging può comportare la perdita delle tracce dell'infezione in tempi relativamente brevi.

Per scongiurare tale perdita è imperativo operare a macchina avviata (*Live Forensics*), introducendo un secondo ostacolo: l'**inaccessibilità dei file a caldo**. L'acquisizione di file critici, come gli *hive* del Registro (NTUSER.DAT), è impedita dal blocco di accesso esclusivo (*exclusive lock*) imposto dal kernel. La normale copia è negata, costringendo l'analista a impiegare complesse tecniche a basso livello (come la *Volume Shadow Copy*) per estrarre le prove.

Isolata la minaccia, l'ultima sfida è la **bonifica del sistema** (*Remediation*). A differenza dei malware tradizionali, risolvibili eliminando l'eseguibile, l'eradicazione *fileless* impone due fasi: il contenimento a caldo (terminazione del processo in RAM) e la rimozione della persistenza. Analisti o sistemi EDR devono eliminare chirurgicamente le stringhe occultate nel Registro o nei task (WMI). Una pulizia parziale comporterebbe infatti il ricaricamento del *payload* al riavvio, vanificando l'intera operazione.

2. Algoritmo di rilevamento per script PowerShell malevoli

2.1 Contesto e Scelte Progettuali

Come analizzato nel capitolo precedente, l'ambiente PowerShell rappresenta lo strumento più sfruttato nell'ambito delle minacce *fileless* [5]. Tuttavia, esso costituisce anche una risorsa fondamentale per la gestione quotidiana delle reti aziendali da parte degli amministratori di sistema. Questo duplice utilizzo rende complessa la progettazione di uno strumento automatizzato di analisi forense: un approccio troppo restrittivo rischia di generare un numero eccessivo di falsi positivi, sovraccaricando i team di sicurezza, mentre un criterio troppo permissivo finisce per non intercettare i comportamenti malevoli nascosti.

Per lo sviluppo del motore di rilevamento si è scelto di focalizzare l'analisi esclusivamente su PowerShell. Questa decisione è motivata sia dalla prevalenza statistica di tale vettore negli attacchi reali, sia dalla disponibilità di dataset sufficientemente estesi per l'addestramento efficace di un modello predittivo. L'intera pipeline software, dalla fase di pre-elaborazione dei dati fino alla classificazione, è stata sviluppata in **Python**, sfruttando le funzionalità offerte dalla libreria `scikit-learn`.

2.2 Classificazione tramite Algoritmo Random Forest

Il processo di classificazione dei comandi estratti nelle categorie «Benigno» o «Maligno» si affida all'algoritmo **Random Forest** [1].

La scelta di questo modello rispetto ad altre architetture è dettata da precise considerazioni ingegneristiche e forensi. Il Random Forest opera come un classificatore d'*ensemble*, combinando le predizioni di molteplici alberi di decisione indipendenti tramite logiche di maggioranza (*bagging*). Questa struttura garantisce un'elevata tolleranza al rumore e una forte resistenza all'overfitting, caratteristiche essenziali quando si analizzano stringhe di testo eterogenee e spesso offuscate.

A differenza dei modelli a rete neurale, storicamente considerati sistemi di tipo *black box* a causa della loro opacità decisionale, il Random Forest offre un livello di interpretabilità decisamente superiore. Attraverso il calcolo nativo dell'importanza delle variabili (*feature importance*), l'algoritmo permette di tracciare quali caratteristiche abbiano maggiormente pesato sul verdetto finale. In informatica forense questo aspetto è cruciale: l'analista non necessita soltanto di un responso binario sulla pericolosità di un comando, ma deve poter isolare gli specifici indicatori (come il richiamo a determinate API o anomalie nella lunghezza del codice) che hanno determinato la segnalazione.

2.2.1 Selezione e Preparazione del Dataset

Per addestrare il modello è stato scelto il dataset pubblico MPSD (*Malicious PowerShell script Dataset*), fornito dal DAS Lab¹.

I dati originali, costituiti da migliaia di script con estensione `.ps1` divisi in varie cartelle, sono stati letti e aggregati in un'unica struttura dati compatibile con la libreria **pandas** tramite uno script Python dedicato. Durante questa fase, si è deciso di salvare il dataset consolidato nel formato **Excel (XLSX)** anziché nel tradizionale formato testuale **CSV**.

¹Il dataset è disponibile su GitHub all'indirizzo: <https://github.com/das-lab/mpsd>

Questa scelta pratica è legata alla sintassi stessa di PowerShell, che fa un uso massiccio del punto e virgola (;) e della virgola (,). In un classico file CSV, questi caratteri avrebbero innescato un problema noto come *delimiter collision*: il sistema avrebbe interpretato erroneamente i simboli del codice come separatori di colonna, spezzando i comandi e corrompendo la tabella. Sfruttando la naturale divisione in celle di Excel, il problema è stato risolto alla radice, associando in modo sicuro ogni comando grezzo alla sua classe di appartenenza (0 per Benigno, 1 per Maligno).

Un punto di forza fondamentale di questo dataset è il suo bilanciamento intrinseco, contenendo un numero quasi uguale di script malevoli e benigni. Questa caratteristica ha permesso di non dover ricorrere a tecniche di riequilibrio artificiale dei dati (come l'*oversampling*), che talvolta rischiano di alterare l'affidabilità delle previsioni del modello.

Infine, prima di iniziare l'analisi, le righe del dataset sono state mescolate in modo casuale (*data shuffling*). Questo passaggio è essenziale per impedire all'algoritmo di memorizzare semplicemente la sequenza dei file, assicurando inoltre che i dati usati per valutare il modello contengano una proporzione corretta e realistica di minacce e script sicuri.

2.2.2 Estrazione delle Caratteristiche (Feature Engineering)

Gli algoritmi di Machine Learning leggono solo numeri, non testo. Quindi è stato necessario trasformare il codice PowerShell in valori matematici. All'inizio si è provato a misurare due aspetti:

- **La lunghezza del comando (*Length*):** I malware spesso usano blocchi di codice eccezionalmente lunghi per ospitare l'intero *payload* o per superare i limiti di analisi di alcuni sistemi di sicurezza.
- **L'Entropia (*Entropy*):** Misura il livello di "disordine" e di imprevedibilità dei caratteri all'interno dello script. I comandi offuscati (ad esempio quelli codificati in Base64 o criptati) presentano un'entropia significativamente più alta rispetto al testo in chiaro. Matematicamente, questo valore viene calcolato applicando la formula dell'Entropia di Shannon:

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$$

dove n rappresenta il numero di caratteri unici presenti nel testo e $p(x_i)$ indica la probabilità (ovvero la frequenza relativa) di comparsa del singolo carattere x_i all'interno dell'intera stringa esaminata.

Facendo delle prove, si è notato che valutare queste metriche separatamente portava a degli errori. Un comando lungo potrebbe essere solo un normale script di manutenzione, e una stringa corta ma disordinata potrebbe essere una password innocua. Il pericolo reale c'è quando un comando è sia molto lungo sia molto disordinato. Per far capire questo concetto all'algoritmo, le due misure sono state unite in una sola variabile chiamata ***Entropy Log Length***.

Inoltre, per capire cosa cerca di fare davvero il codice, è stata creata la variabile ***Danger Density*** (Densità di Pericolosità). Questa metrica calcola un punteggio di rischio sommando la presenza di parole sospette, comandi pericolosi (come ***Invoke-Expression***) e tentativi di aggirare gli antivirus di Windows.

Selezione delle Variabili Comportamentali

Oltre alle metriche statistiche generali, il modello è stato arricchito con variabili booleane e densità di rischio progettate per intercettare pattern d'attacco specifici. La scelta di queste *feature* è scaturita da un'analisi sistematica delle tecniche di *obfuscation* e delle API di Windows più abusate.

- **Upper Case Ratio:** Questa variabile calcola il rapporto tra lettere maiuscole e la lunghezza totale del comando. È stata introdotta per contrastare la tecnica di *Case Randomization* (es. **p0wErShElL**): gli attaccanti alternano casualmente il registro dei caratteri per evadere i sistemi di rilevamento basati su firme statiche *case-sensitive*. Una distribuzione non standard delle maiuscole è un forte indicatore di codice generato artificialmente o manipolato.
- **Has IEX (Invoke-Expression):** Identificata come una delle *feature* più critiche, traccia la presenza del comando **iex** o dei suoi alias. È il vettore fondamentale dei malware *fileless*, poiché permette di valutare una stringa di testo come codice eseguibile, consentendo al *payload* di risiedere esclusivamente nella memoria RAM senza mai toccare il disco rigido.

- **Has DLLImport e API Calls:** Per rilevare attacchi avanzati di *Process Injection*, sono stati inseriti sensori per le chiamate a librerie esterne (tramite `DllImport`) e API specifiche come `VirtualAlloc` o `CreateRemoteThread`. Queste funzioni sono raramente utilizzate in script di amministrazione standard, ma sono essenziali per i malware che tentano di iniettare codice malevolo all'interno di processi legittimi già in esecuzione.
- **Has Encoded e Has Persistence:** Mentre la prima identifica l'uso di Base64 per nascondere la reale natura del comando, la seconda monitora tentativi di scrittura nel Registro di Sistema o la creazione di *Scheduled Tasks*. Queste variabili sono state scelte per mappare la capacità del malware di sopravvivere ai riavvii della macchina, un requisito fondamentale per le infezioni a lungo termine.

L'integrazione di questi sensori permette al modello **Random Forest** di non limitarsi a un'analisi superficiale, ma di pesare la gravità di ogni singola azione all'interno di una variabile aggregata denominata ***Danger Density***. In questo modo, lo *score* finale non dipende da un singolo termine sospetto, ma dalla convergenza di più segnali che indicano inequivocabilmente un intento malevolo.

Categorizzazione delle Feature

Alla fine di questo processo di pre-elaborazione, per ogni script sono state estratte **23 variabili** (*feature*), divise in tre categorie matematiche principali, come mostrato nella Tabella 2.1:

Tipo di Dato	Descrizione e Variabili
Numeri Decimali	Valutano le anomalie statistiche nel testo: • entropy_log_length: Indica quanto il codice è lungo e disordinato. • upper_case_ratio: Valuta l'alternanza anomala tra lettere maiuscole e minuscole, usata per confondere gli antivirus.
Valori Booleani	20 variabili che si attivano (1) in presenza di specifici pattern d'attacco o elusione: • <i>Esecuzione e Occultamento</i>: has_iex, has_encoded e relative varianti. • <i>Interazione a Basso Livello</i>: has_dllimport, api_virtualalloc, api_createremotethread e affini. • <i>Persistenza e Azioni Critiche</i>: has_persistence e altri indicatori legati alla manipolazione del sistema (es. file system o processi).
Somma Matematica	danger_density : È la somma aggregata di tutti i valori booleani precedenti attivati in uno script, fungendo da punteggio finale di pericolosità.

Tabella 2.1. Struttura delle 23 feature estratte dalla stringa del comando.

Importanza delle Variabili (Feature Importance)

L'efficacia di questo lavoro di preparazione dei dati è stata confermata analizzando la *Feature Importance* fornita dal Random Forest. Questa metrica dice esattamente quanto ogni variabile ha pesato sulle decisioni prese dall'algoritmo.

Come si vede nella Tabella 2.2, le due variabili create appositamente per questo progetto guidano da sole quasi il 67% delle decisioni:

Feature	Importanza sul Modello (%)
danger_density	47.52
entropy_log_length	19.23
has_iex	11.06
has_encoded	8.60
upper_case_ratio	7.37
has_reflection	1.30
has_api_calls	1.20
<i>Altre 15 feature (sommate)</i>	<i>3.72</i>

Tabella 2.2. Classifica delle variabili che hanno influenzato di più il modello.

I dati dimostrano che:

- La `danger_density` (47.52%) è in assoluto l'indicatore principale. Questo fa capire che una singola parola sospetta non basta per etichettare un file come virus; il modello ha imparato che la vera minaccia è l'accumulo di tante azioni sospette nello stesso comando.
- L'importanza di `entropy_log_length` (19.23%) conferma che unire lunghezza ed entropia ha funzionato, aiutando il modello a trovare i codici nascosti senza far scattare allarmi per i normali script di sistema.
- Le variabili `has_iex` (11.06%) e `has_encoded` (8.60%) confermano che l'uso della codifica Base64 e l'esecuzione diretta in memoria sono ancora le tecniche preferite dai malware fileless.

2.2.3 Ottimizzazione del Modello e Metrica F2

Di solito, per valutare un modello si cerca di massimizzare l'Accuratezza totale. Tuttavia, nella sicurezza informatica questo approccio è pericoloso: un **Falso Negativo** (un malware non rilevato) crea danni molto più gravi di un Falso Positivo (un falso allarme su un file innocuo).

Per questo motivo, l'algoritmo è stato ottimizzato cercando di massimizzare un'altra metrica, il **F2-Score**:

$$F_{\beta} = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{(\beta^2 \cdot \text{Precision}) + \text{Recall}}$$

Impostando il valore $\beta = 2$, si dice matematicamente al modello di dare un peso doppio al *Recall* (la capacità di trovare tutte le minacce) rispetto alla *Precision* (la capacità di non fare falsi allarmi).

Facendo diverse prove con la tecnica della *Grid Search*, si è notato che aggiungere troppi alberi decisionali appesantiva il sistema senza migliorarne le previsioni. Il miglior risultato per l’F2-Score è stato raggiunto creando una foresta di 54 alberi (`n_estimators=54`) con una profondità massima di 30 livelli (`max_depth=30`).

2.2.4 Validazione e Risultati Finali

Per verificare che il modello sia in grado di riconoscere correttamente anche script mai visti in precedenza, è stata utilizzata la tecnica della *Stratified 5-Fold Cross-Validation*. Questo metodo suddivide il dataset in 5 parti: a turno, 4 parti vengono usate per l’addestramento e la restante per il test.

Le prestazioni percentuali ottenute in ogni fase, inclusi gli indici F1 e F2, sono riassunte nella Tabella 2.3.

Test	Accur. (%)	Prec. (%)	Rec. (%)	F1 (%)	F2 (%)
1	98.57	98.81	98.34	98.57	98.43
2	98.69	98.69	98.69	98.69	98.69
3	98.34	99.04	97.62	98.33	97.90
4	98.22	99.03	97.39	98.20	97.71
5	98.57	98.92	98.22	98.57	98.36
Media	98.48	98.90	98.05	98.47	98.22

Tabella 2.3. Risultati della validazione incrociata sulle 5 partizioni del dataset.

Il classificatore ha raggiunto un’Accuratezza media del 98.48%. L’uso dell’F2-Score come metrica di riferimento ha permesso di dare priorità al Recall (98.05%),

assicurando che la quasi totalità dei malware venga identificata correttamente. La Precisione media (98.90%) indica inoltre una bassissima incidenza di falsi allarmi.

Per analizzare il peso reale di queste percentuali, i dati assoluti estratti dalle matrici di confusione di ogni singolo test sono stati raggruppati nella Tabella 2.4.

Test	Veri Negativi (TN)	Falsi Positivi (FP)	Falsi Negativi (FN)	Veri Positivi (TP)
1	831	10	14	827
2	829	11	11	831
3	832	8	20	822
4	832	8	22	820
5	831	9	15	826
Totale	4155	46	82	4126

Tabella 2.4. Valori assoluti della classificazione per ogni fold e somma totale sull'intero dataset.

Guardando i dati aggregati, l'efficacia del *Random Forest* risulta evidente: su un totale di 8409 script analizzati, il modello ha bloccato 4126 *payload* malevoli fallendone solamente 82 (FN). Allo stesso tempo, ha disturbato l'analista con soli 46 falsi allarmi (FP) su oltre 4100 comandi legittimi.

2.2.5 I Limiti dell'Intelligenza Artificiale e il Sistema Ibrido

Nonostante gli ottimi numeri, affidare la sicurezza solo all'Intelligenza Artificiale può essere rischioso. Alcuni script creati dai sistemisti sono molto complessi e potrebbero far scattare l'allarme per errore. Inoltre, i criminali inventano continuamente nuovi modi per nascondere il codice, costringendo ad addestrare di nuovo il modello.

Per risolvere questi problemi pratici, nell'applicazione finale il Random Forest non lavora da solo, ma è stato inserito in un sistema di controlli a tre passaggi:

1. **Controllo dell'Hash (Whitelist):** Per prima cosa si calcola l'impronta digitale (SHA-256) dello script. Se corrisponde a un file aziendale conosciuto

e sicuro, viene approvato subito, azzerando i falsi positivi per le procedure normali.

2. **Machine Learning:** Se lo script è sconosciuto, viene analizzato dal Random Forest, che calcola la probabilità che si tratti di un malware basandosi su quello che ha imparato.
3. **Regole Euristiche di Sicurezza:** Se il modello matematico è incerto, ma lo script sta cercando di fare danni evidenti (come usare il comando `vssadmin` per cancellare i backup di sistema), un controllo a regole fisse scavalca l'IA e blocca l'operazione. Questo garantisce che gli attacchi più pericolosi, come i ransomware, vengano fermati in ogni caso.

3. Implementazione del Tool di Analisi Forense

Dopo aver definito e addestrato il modello di Machine Learning, è stato necessario applicare questa tecnologia a uno scenario pratico. Per questo motivo, è stato sviluppato un tool automatizzato di analisi forense, scritto interamente in linguaggio Python.

L'obiettivo del software è supportare l'analista nell'identificazione di comandi malevoli, estraendo e analizzando in modo rapido gli specifici file di sistema che conservano le tracce (artefatti) tipiche di un'infezione *fileless*.

Il programma si presenta con un'interfaccia grafica moderna e intuitiva, realizzata tramite le librerie `tkinter` e `customtkinter`. Per garantire la massima flessibilità operativa, il tool offre due modalità di utilizzo: l'analista può scegliere se avviare l'acquisizione automatica dei log direttamente dal sistema in uso, oppure procedere al caricamento manuale. Questa seconda opzione risulta fondamentale qualora si debbano esaminare file estratti in precedenza da un computer esterno compromesso.

Essendo uno strumento progettato per interagire in profondità con file di sistema protetti e registri critici, l'avvio dell'applicazione richiede tassativamente l'esecuzione con privilegi di amministratore (*Run as Administrator*). In assenza di tali permessi, i meccanismi di sicurezza di Windows bloccherebbero le operazioni di lettura, inibendo la fase di acquisizione.

Una volta caricati i dati, il software avvia il motore di classificazione: carica in memoria il modello Random Forest descritto nel capitolo precedente e analizza ogni singola stringa di codice trovata.

Al termine dell'elaborazione, l'esito dell'analisi viene presentato attraverso una

tabella interattiva. I comandi vengono elencati in ordine decrescente di pericolosità, mettendo subito in risalto le minacce più critiche. Per facilitare il lavoro investigativo, ogni riga fornisce all'analista tutte le informazioni di contesto necessarie, tra cui la data di esecuzione, l'origine dell'artefatto (il file di provenienza) e il comando esatto utilizzato.

Al fine di garantire la totale trasparenza metodologica e la riproducibilità dell'intero ecosistema di difesa, il codice sorgente del progetto è stato rilasciato pubblicamente. All'interno del repository GitHub dell'autore¹ è possibile visionare l'intera architettura software: non solo il codice del tool forense ad interfaccia grafica descritto in questo capitolo, ma anche l'implementazione completa del modello predittivo *Random Forest*, le regole del modulo euristico e gli script per l'estrazione delle feature.

3.1 Acquisizione degli Artefatti di Sistema

Il primo passo operativo del framework consiste nella raccolta automatizzata delle evidenze digitali (artefatti). Invece di eseguire una scansione completa del disco rigido, operazione che richiederebbe ore e rallenterebbe le indagini, il modulo di acquisizione (`acquisition.py`) interroga in modo mirato tre sorgenti specifiche dell'ecosistema Windows.

Seguendo le *best practice* dell'informatica forense, lo scopo di questa fase non è ancora leggere i dati, ma isolare i file originali effettuandone una copia esatta in un'area di lavoro temporanea, per non rischiare di alterarli o corromperli durante la successiva analisi.

3.1.1 Cronologia dei Comandi (Console History)

A partire dalla versione 5.0 di PowerShell, Windows registra automaticamente ogni comando digitato dall'utente all'interno di un file di testo denominato `ConsoleHost_history.txt`, situato tipicamente nel percorso `AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine` del profilo

¹Il codice sorgente del progetto è consultabile al seguente indirizzo: <https://github.com/nbellomo506/cyber-thesis>

utente.

Il tool automatizza il recupero di questo file risolvendo dinamicamente il percorso tramite le variabili d'ambiente di Windows (`USERPROFILE`) e copiandolo nello spazio temporaneo dell'analista. Sebbene sia una fonte di dati molto ricca, presenta due limiti forensi intrinseci. In primo luogo, un attaccante esperto può facilmente cancellare le proprie tracce svuotando la cronologia (ad esempio, usando il comando `Clear-History`). In secondo luogo, trattandosi di un semplice file testuale (*plain text*), le singole righe di comando non sono associate ad alcuna marcatura temporale (*timestamp*). L'unica informazione cronologica direttamente ricavabile è dettata dai metadati del file stesso (come la data di ultima modifica), che documenta esclusivamente il momento in cui è stato eseguito l'ultimo comando in assoluto.

A causa dell'impossibilità di stabilire una *timeline* precisa delle azioni pregresse, questa sorgente viene utilizzata dal software come primo livello di indagine, i cui risultati devono essere necessariamente incrociati con artefatti più strutturati.

3.1.2 Event Logging Avanzato (Event ID 4104)

La fonte di verità assoluta per l'analisi dei malware fileless è lo *Script Block Logging* di Windows. Quando abilitata, questa funzione registra nel visualizzatore eventi (*Event Viewer*) l'esatto blocco di codice PowerShell un attimo prima che venga eseguito dalla CPU, catalogandolo con l'**Event ID 4104**. L'importanza critica di questo log risiede nella sua capacità di catturare il codice in chiaro ("de-offuscato") subito dopo che il malware lo ha decodificato per poterlo eseguire.

Per estrarre questi dati in modo sicuro, l'applicazione non si appoggia a librerie esterne di terze parti, ma adotta una tecnica di *Living off the Land* (LotL). Attraverso il modulo `subprocess` di Python, il software invoca `wevtutil`, un'utilità nativa di Windows usata dagli amministratori di sistema. Inviando il comando `wevtutil ep1`, il framework esporta una copia statica e pulita dell'intero registro `Microsoft-Windows-PowerShell/Operational` in un file temporaneo `.evtx`. Trattandosi di log di sicurezza, questa operazione richiede tassativamente l'esecuzione del software con privilegi di Amministratore.

3.1.3 L'artefatto di Persistenza: NTUSER.DAT e Live Forensics

Come discusso nei capitoli precedenti, i malware fileless abusano del Registro di Sistema per garantirsi la persistenza e nascondere i propri *payload*. Poiché le impostazioni utente sono salvate fisicamente sul disco nel file nascosto NTUSER.DAT, il framework ne prevede l'estrazione completa.

L'acquisizione fisica di questo file è un passaggio investigativo cruciale [2]: a differenza di una semplice interrogazione del registro in tempo reale tramite API di sistema (che mostra esclusivamente lo stato attuale e "vivo" delle chiavi), l'analisi forense del file NTUSER.DAT permette di recuperare tracce storiche, dati frammentati e chiavi alterate di recente, conservando le prove dell'infezione per un tempo nettamente superiore.

Tuttavia, l'estrazione di questo artefatto introduce una notevole sfida tecnica: a sistema avviato, il kernel di Windows mantiene un blocco di accesso esclusivo (*exclusive lock*) sul file, impedendo le normali operazioni di copia. Poiché spegnere il sistema per aggirare il blocco (*Dead Forensics*) comporterebbe la perdita irrimediabile delle prove volatili presenti nella RAM, il tool implementa un avanzato meccanismo di *Live Forensics* basato sul *Volume Shadow Copy Service* (VSS).

Per garantire la massima portabilità ed evitare l'uso di librerie esterne, il software utilizza un approccio *Living off the Land* (LotL), orchestrando in background strumenti nativi di Windows. Il processo automatizzato aggira il blocco di sistema in quattro passaggi:

1. **Preparazione:** Il programma controlla di avere i permessi di Amministratore e individua in automatico in quale cartella si trova il profilo dell'utente.
2. **Fotografia del disco (Snapshot):** Sfruttando i comandi nativi di Windows, il software scatta un'istantanea invisibile dell'intero disco rigido. Ci vuole solo una frazione di secondo e il sistema operativo non viene interrotto.
3. **Esposizione del Percorso (Symlink Execution):** Poiché Windows alloca le *shadow copy* in directory protette dal kernel, l'accesso diretto ai file risulta strutturalmente inibito. Il framework aggira l'isolamento configurando un

punto di giunzione logico tramite *symlink* nel disco principale (C:). Tale livello di astrazione permette sia all'analista umano sia ai componenti di *parsing* automatizzati di navigare e interrogare il punto di ripristino con le medesime modalità di una directory standard.

4. **Estrazione dell'Artefatto e Smontaggio del Volume:** Svincolato il file NTUSER.DAT dalle restrizioni di accesso esclusivo, il tool ne esegue l'esportazione sicura. Al termine dell'acquisizione, le routine di pulizia provvedono all'eliminazione del collegamento logico e alla distruzione dello *snapshot* del disco.

3.2 Parsing e Normalizzazione dei Dati

Una volta che i file contenenti gli artefatti sono stati isolati e copiati nello spazio temporaneo di lavoro, il framework deve uniformare informazioni provenienti da formati strutturalmente opposti (testo semplice, log binari e hive di registro). Il modulo `parsers.py` svolge la fondamentale funzione di "traduttore", analizzando ogni file ed estraendo un dizionario standardizzato contenente tre informazioni chiave: il *timestamp*, l'origine (sorgente) e il comando PowerShell grezzo (payload).

3.2.1 Elaborazione dell'Event Log (EVTX)

L'elaborazione più complessa riguarda i log di sistema (`.evtx`), i quali sono archiviati in formato binario. Il parser utilizza la libreria `python-evtx` per decodificare dinamicamente i blocchi binari, trasformandoli in una struttura XML (*eXtensible Markup Language*) navigabile.

L'algoritmo ispeziona l'albero XML alla ricerca del nodo `System`, filtrando esclusivamente gli eventi etichettati con `EventID 4104` (Script Block Logging). Quando viene trovato un *match*, il software naviga nel nodo `EventData` per estrarre la stringa contenuta nel campo `ScriptBlockText`. In questa fase, il parser svolge anche un'importante funzione di *Timeline Reconstruction*: estrae l'attributo `SystemTime`, ripulisce i millisecondi e il fuso orario (*Z-time*), e lo normalizza nel formato europeo (GG-MM-AAAA). Questo passaggio è critico per permettere all'analista di ricostruire la catena temporale dell'attacco.

3.2.2 Ottimizzazione della Leggibilità per l'Analisi Manuale

I comandi malevoli estratti dai log e dal registro, specialmente se precedentemente offuscati o codificati in Base64, si presentano molto spesso come "monoliti": un'unica, lunghissima riga di testo in cui le istruzioni sono separate solo da punti e virgola. Questo formato, pur essendo perfettamente digeribile per il Machine Learning, risulta incomprensibile per un analista umano.

Per risolvere questo problema di *User Experience* (UX) investigativa, all'interno del modulo di orchestrazione (`app.py`) è stata implementata una funzione di *Code Beautifying*. L'algoritmo effettua il parsing della stringa grezza ed esegue una formattazione automatica basata sulla sintassi di PowerShell:

- Suddivide le istruzioni andando a capo in corrispondenza del terminatore di linea (;) e del carattere di *pipe* (|).
- Calcola dinamicamente il livello di indentazione contando l'apertura e la chiusura delle parentesi graffe ({ e }).

Il risultato è un codice sorgente strutturato, ri-indentato e facilmente leggibile. Questa formattazione viene generata dal sistema in modalità *on-demand*, ovvero esclusivamente nel momento in cui l'analista decide di ispezionare visivamente i dettagli di uno specifico comando tramite l'interfaccia grafica.

3.3 Design dell'Interfaccia e Sincronizzazione dei Processi

Un'interfaccia utente (GUI) deve essere sempre veloce e reattiva, specialmente in un software forense. Analizzare migliaia di log o interrogare il modello di Intelligenza Artificiale richiede però un grande sforzo da parte del processore. Se il programma provasse a fare questi calcoli pesanti e, contemporaneamente, a gestire i clic dell'utente sullo stesso "binario" (il flusso principale), l'interfaccia si bloccherebbe inesorabilmente, mostrando il classico errore "Il programma non risponde".

Per garantire la stabilità dell'applicativo, il codice (all'interno del file `app.py`) è stato strutturato usando la tecnica del *Multithreading*, ovvero l'esecuzione di più compiti in parallelo.

Tramite la funzione dedicata `run_in_background`, il lavoro viene smistato in tempo reale su due strade diverse:

- **Il flusso principale (Main UI Thread):** Si occupa esclusivamente di disegnare l'interfaccia e ascoltare i clic del mouse. Essendo completamente svincolato dai calcoli pesanti, garantisce che la finestra rimanga sempre fluida.
- **Il flusso secondario (Background Worker Thread):** Viene instradato in modo asincrono solo per l'esecuzione di task ad alto carico computazionale, quali l'estrazione massiva dei file o l'analisi predittiva, aggiornando l'interfaccia grafica unicamente al termine del processo.

Avere due flussi che lavorano in parallelo risolve il problema dei blocchi, ma introduce un rischio informatico noto come *Race Condition*. Cosa succederebbe se l'utente premesse accidentalmente altri bottoni mentre il flusso secondario sta ancora elaborando i log? I dati potrebbero sovrasciversi, corrompendo l'intera indagine forense.

Per proteggere l'integrità dei dati, è stato implementato un **Overlay di Protezione**. Ogni volta che parte un lavoro in *background*, l'interfaccia grafica viene istantaneamente coperta da un pannello scuro a schermo intero. Questo *Loader* agisce come uno scudo fisico:

1. **Blocca i clic indesiderati:** Rende inattaccabili i bottoni sottostanti, impedendo all'utente di lanciare comandi involontari finché il calcolo non è terminato.
2. **Rassicura l'analista:** Mostra messaggi animati (come "Elaborazione ML in corso..."), confermando all'investigatore che il programma non si è bloccato, ma sta lavorando.

Quando il flusso secondario termina la sua analisi, il pannello scuro scompare automaticamente e l'interfaccia restituisce i risultati finali, garantendo un'esperienza d'uso fluida e a prova di errore.

3.3.1 Progettazione della Dashboard Principale

A completamento dell'architettura asincrona, l'interfaccia utente è stata progettata seguendo i principi di chiarezza e accessibilità, fondamentali per gli strumenti di *Incident Response* dove l'operatore deve poter agire rapidamente senza distrazioni.

La schermata principale (*Dashboard*) funge da cruscotto operativo e si divide in due macro-aree funzionali: il **Pannello di Acquisizione** e il **Motore di Analisi**.

Il Pannello di Acquisizione guida l'analista nel caricamento dei tre artefatti forensi supportati. Come riassunto nella Tabella 3.1, per ogni artefatto l'operatore ha a disposizione un feedback visivo sullo stato corrente e due percorsi operativi distinti: l'importazione manuale o l'estrazione automatica dal sistema in uso.

PowerShell Commands Forensic Analyzer			
Artefatto Forense	Stato	Azione Manuale	Azione Automatica
1. Cronologia PowerShell (File TXT)	Acquisito (Auto)	Sfoglia...	Acquisisci
2. Eventi di Sistema (Log EVTX - EID 4104)	Nessun file	Sfoglia...	Esporta
3. Registro di Sistema (Hive NTUSER.DAT)	Nessun file	Sfoglia...	Estrai
Acquisizione Massiva (Estrazione combinata)	Pronto	-	Estrai Tutto

Tabella 3.1. Struttura logica e opzioni di interazione del Pannello di Acquisizione. Il primo artefatto risulta già correttamente estratto dal sistema.

Il software fornisce un *feedback* di stato dinamico, aggiornando la colonna centrale (da "Nessun file" a "Acquisito/Pronto" evidenziato in verde) per indicare quali file sono pronti. Eventuali mancanze di privilegi (come l'assenza dei diritti di Amministratore richiesti per il Volume Shadow Copy o per i log EVTX) vengono intercettate restituendo un alert visivo all'utente.

Infine, tramite il pulsante "**Avvia Analisi Integrata**", il sistema raccoglie i dati da tutte le fonti caricate con successo, li normalizza tramite i *parser* dedicati e li invia in blocco al modello di *Machine Learning* per la valutazione comportamentale.

3.4 Esempio di Risultati dell'Analisi

Per illustrare concretamente il potenziale investigativo dell'architettura software sviluppata, vengono riportati di seguito alcuni esempi significativi di log analizzati dal sistema.

È importante sottolineare che, per ottimizzare il flusso di lavoro dell'analista forense ed evitare il sovraccarico di informazioni (*information overload*), l'interfaccia del tool è stata progettata per filtrare i risultati in base alla loro pericolosità. Nello specifico, il sistema visualizza esclusivamente i comandi che il motore di classificazione Random Forest valuta con una probabilità di malignità (Score) superiore al 50%, etichettandoli come "*Sospetti*". Qualora tale valore superi la soglia dell'80%, il comando viene immediatamente contrassegnato come "*Critico*", segnalando un'altissima probabilità di attacco avvenuto o in corso.

In questo modo, l'operatore può ignorare le migliaia di istruzioni PowerShell legittime eseguite dal sistema e concentrarsi sui pochi eventi che richiedono un'ispezione immediata. La Tabella 3.2 e il seguente elenco descrivono tre scenari reali rilevati durante la fase di test:

Tabella 3.2. Esempio dei risultati riassuntivi mostrati nella dashboard del tool.

Data ed Ora	Livello (Score)	Sorgente	Anteprima Comando
09-05-2026 14:32	CRITICO (98%)	Event Log	powershell -enc JABzAD0...
08-05-2026 09:15	CRITICO (92%)	Registro	IEX (New-Object Net...
09-05-2026 14:30	SOSPETTO (75%)	Cronologia	powershell -Execution...

Analizzando nel dettaglio i *payload* grezzi estratti dal tool, possiamo notare le tre casistiche tipiche affrontate dal sistema:

- **Il comando cifrato (Score: 98%):** Il malware ha tentato di nascondere il proprio intento utilizzando il parametro `-enc` (EncodedCommand) e la codifica Base64. Il tool ha estratto l'intero artefatto dai log, classificandolo come critico a causa dell'elevata entropia logaritmica e della struttura offuscata. Il comando originale rilevato è il seguente:

```
powershell.exe -nop -w hidden -enc WwBSAGUAZgB...
```

Procedendo con la decodifica forense della stringa da Base64 a testo in chiaro, emerge il reale comportamento del *payload*: non un semplice download, ma un *AMSI Bypass*. In pratica, il comando sfrutta la tecnica della *Reflection* per accedere alle variabili interne di PowerShell, individuando il flag di errore `amsiInitFailed` e forzandolo al valore `$true`. Così facendo, l'Antimalware Scan Interface (AMSI) viene indotta in errore e l'antivirus viene letteralmente "accecato", rendendo invisibili ai controlli tutti i successivi comandi malevoli eseguiti in quella specifica sessione.

```
[Ref].Assembly.GetType(  
'System.Management.Automation.AmsiUtils')  
.GetField('amsiInitFailed','NonPublic,Static')  
.SetValue($null,$true)
```

- **Il download in memoria (Score: 92%):** All'interno del Registro di Sistema è stato individuato un **dropper in-memory** (o caricatore volatile). In questo scenario, l'elevato punteggio di rischio è dovuto all'impiego dell'istruzione IEX (*Invoke-Expression*) concatenata a una richiesta di rete. Questa tecnica permette di scaricare uno script malevolo da un server remoto e di "iniettarlo" direttamente nell'interprete PowerShell senza che venga mai creato un file sul disco rigido. Poiché il codice risiede esclusivamente nella RAM, l'attacco riesce a eludere i controlli dei software antivirus basati sulla scansione dei file statici.

```
IEX (New-Object Net.WebClient)  
.DownloadString(  
'http://10.10.14.5/payload.ps1')
```


- **L'aggiramento delle policy (Score: 75%):** Questo comando altera esplicitamente i permessi di esecuzione di PowerShell. Viene etichettato come "Sospetto" (e non Critico) in quanto potrebbe rappresentare sia una fase di preparazione di un attacco, sia una legittima operazione di aggiornamento da parte di un amministratore.

```
powershell.exe -ExecutionPolicy Bypass  
-WindowStyle Hidden -File  
C:\Users\Admin\AppData\Local\Temp\update.ps1
```


4. Conclusioni e Sviluppi Futuri

4.1 Conclusioni

Il presente lavoro di tesi ha affrontato una delle minacce più insidiose e in rapida evoluzione nel panorama contemporaneo della cybersecurity: il *fileless malware*. L'abilità di queste minacce di operare esclusivamente all'interno della memoria volatile (RAM) e di abusare di strumenti di amministrazione legittimi del sistema operativo, in particolare PowerShell, rappresenta una sfida critica per i tradizionali meccanismi di difesa basati sulle firme statiche del file system o scansione dello stesso.

Per rispondere a questa problematica, è stato progettato e implementato un framework investigativo orientato alla *Digital Forensics* e all'*Incident Response*. L'architettura sviluppata non si limita alla raccolta di artefatti storici come la *Console History* o l'hive di registro NTUSER.DAT, ma estrae il massimo valore informativo dai log operativi avanzati di Windows, nello specifico lo *Script Block Logging* (Event ID 4104), che consente di intercettare il codice nel momento esatto della sua esecuzione in memoria, superando qualsiasi livello di offuscamento preventivo.

Il cuore tecnologico del sistema, basato sull'algoritmo di Machine Learning *Random Forest*, ha dimostrato sul campo l'efficacia dell'approccio analitico proposto. Grazie all'ingegnerizzazione di 23 feature numeriche e comportamentali — tra cui metriche introdotte specificamente per questo scenario come la *Danger Density* e la *Entropy Log Length* — il modello ha raggiunto risultati eccellenti in fase di validazione incrociata (*5-Fold Cross-Validation*), registrando un'accuratezza media del 98,48% e un Recall del 98,05%.

L'ottimizzazione del classificatore attraverso la metrica F2-Score ha permesso di ridurre drasticamente l'impatto dei falsi negativi, un requisito fondamentale in questo contesto, dove l'omissione di una minaccia reale può compromettere l'intera infrastruttura. Su un dataset complessivo di oltre 8400 script, il sistema ha correttamente identificato 4126 payload malevoli, generando appena 46 falsi positivi.

Infine, l'integrazione del modello predittivo all'interno di un'architettura di controllo ibrida a tre stadi (composta da Whitelist, Machine Learning e Regole Euristiche) e l'interfaccia grafica asincrona realizzata dimostrano la fattibilità ingegneristica del progetto, configurandolo come un solido strumento di supporto decisionale capace di automatizzare l'analisi e il filtraggio dei log e di ridurre sensibilmente i tempi di investigazione per gli analisti della sicurezza.

4.2 Sviluppi Futuri

Nonostante gli ottimi risultati conseguiti, il framework presenta ampi margini di evoluzione e potenziamento, integrabili in successive fasi di ricerca e sviluppo:

- **Estrazione e Parsing del WMI:** Attualmente, l'applicativo non esegue l'ispezione diretta del *repository* fisico del WMI (il file `OBJECTS.DATA`), una scelta dettata dalla complessità della struttura proprietaria del database, ma che lascia una potenziale zona d'ombra per le tecniche di persistenza avanzate basate su *Event Filter* e *Event Consumer*. L'implementazione futura di un estrattore WMI consentirebbe di acquisire tali oggetti logici in modalità *Live Forensics*, normalizzare gli script iniettati e sottoporli al medesimo motore di *Machine Learning* già validato in questo lavoro di tesi.
- **Integrazione Real-Time ed EDR:** Attualmente il tool opera secondo una logica di analisi forense post-evento o di filtraggio su log già esportati. Uno sviluppo futuro di grande rilievo prevede la trasformazione dell'applicativo in un agente residente in background, strutturato come un sistema EDR (*Endpoint Detection and Response*) minimale, capace di agganciarsi direttamente ai canali di *Event Tracing for Windows* (ETW) per bloccare i processi PowerShell malevoli in tempo reale prima che possano completare la catena di infezione.

- **Estensione Multi-Linguaggio:** Sebbene PowerShell rappresenti il vettore principale negli ambienti Windows, le tecniche *Living off the Land* coinvolgono sempre più spesso altri interpreti. Il motore di estrazione delle feature potrebbe essere esteso e riaddestrato per classificare script scritti in Python, macro VBA, file batch (.bat/.cmd) o script Bash in ambienti Linux, standardizzando il processo di classificazione delle minacce testuali.
- **Ottimizzazione del Modello e Algoritmi Alternativi:** Il modello Random Forest ha garantito un eccellente bilanciamento tra accuratezza e tempi di computazione. Uno sviluppo futuro naturale prevede l'estensione della ricerca attraverso uno studio comparativo con un ventaglio più ampio di algoritmi di *Machine Learning*. Nello specifico, le prestazioni del sistema potrebbero essere messe a confronto con modelli classici quali *Support Vector Machines* (SVM), *K-Nearest Neighbors* (KNN), *Naive Bayes* e *Logistic Regression*, per poi valutare architetture più complesse basate su *Gradient Boosting* o sulla combinazione pesata di più classificatori. Infine, prendendo spunto da recenti approcci proposti in letteratura [3], un'ulteriore via di sviluppo potrebbe includere la sperimentazione con le Reti Neurali Convoluzionali (CNN) applicate all'analisi dei malware [4].
- **Ampliamento del Dataset e Dipendenza Euristica:** Sebbene il framework sia stato validato su un dataset esteso, si riscontra in letteratura una generale scarsità di campioni PowerShell associati a ceppi di *ransomware fileless*. Attualmente, il sistema compensa in modo efficace questa criticità attraverso un modulo euristico di *override* che forza un punteggio di allarme massimo in presenza di specifici pattern distruttivi. Tuttavia, l'integrazione di nuovi payload in fase di addestramento permetterebbe di delegare tale riconoscimento interamente al motore predittivo, riducendo la dipendenza dalle regole statiche e migliorando la sensibilità del classificatore verso minacce emergenti.
- **Evoluzione verso una Classificazione Multi-Classe:** L'attuale algoritmo è stato concepito come un classificatore binario, addestrato per discriminare esclusivamente tra script benigni e malevoli. Un importante sviluppo

ingegneristico consisterebbe nell'espansione del modello verso un'architettura multi-classe. Questo permetterebbe non solo di rilevare la presenza di un'infezione, ma anche di categorizzare automaticamente la specifica tipologia di minaccia (*Ransomware*, *Trojan*, *Spyware*, *Worm*, *Cryptominer*). Tale livello di granularità fornirebbe agli analisti forensi un contesto tattico immediato, ottimizzando radicalmente le procedure di *Incident Response* e la prioritizzazione degli interventi.

Bibliografia

- [1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [2] Harlan Carvey. *Windows Registry Forensics: Advanced Digital Forensic Analysis of the Windows Registry*. Syngress, 2nd edition, 2016.
- [3] Christian Catalano, A. Chezzi, M. Angelelli, and F. Tommasi. Deceiving ai-based malware detection through polymorphic attacks. *Computers in Industry*, 143:103751, 2022.
- [4] cridin1. Malware-classification-cnn, 2023. Repository GitHub. <https://github.com/cridin1/malware-classification-CNN>.
- [5] Yong Fang, Xiangyu Zhou, and Cheng Huang. Effective method for detecting malicious powershell scripts based on hybrid features. *Neurocomputing*, 448:30–39, 2021.
- [6] Matt Graeber. Abusing windows management instrumentation (wmi) to build a persistent, asynchronous, and fileless backdoor. In *Black Hat USA*. Black Hat, 2015.
- [7] Danny Hendler, Shay Kels, and Amir Rubin. Amsi-based detection of malicious powershell code using contextual embeddings. In *Proceedings of the 2018 ACM on Asia Conference on Computer and Communications Security (ASIACCS)*, pages 679–693. ACM, 2018.
- [8] Karen Kent, Suzanne Chevalier, Tim Grance, and Hung Dang. Guide to integrating forensic techniques into incident response. Technical Report Special Publication (NIST SP) 800-86, National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, 2006.
- [9] LOLBAS Project. Living off the land binaries, scripts and libraries, 2024. <https://lolbas-project.github.io/>.

- [10] Microsoft. Informazioni sul registro di sistema di windows per gli utenti esperti, Gennaio 2025. <https://learn.microsoft.com/it-it/troubleshoot/windows-server/performance/windows-registry-advanced-users>.
- [11] Microsoft Corporation. Antimalware scan interface (amsi). <https://learn.microsoft.com/en-us/windows/win32/amsi/antimalware-scan-interface-portal>, 2015. Ultimo accesso: 20 Maggio 2026.
- [12] MITRE ATT&CK. Boot or logon autostart execution: Registry run keys / startup folder, 2024. <https://attack.mitre.org/techniques/T1547/001/>.
- [13] MITRE ATT&CK. Windows management instrumentation, 2024. Technique T1047. <https://attack.mitre.org/techniques/T1047/>.
- [14] Sudhakar and Sushil Kumar. An emerging threat fileless malware: a survey and research challenges. *Cybersecurity*, 3(1), 2020.